TEEMATE · 27 OCTOBER 2026

How TeeMate is built

What the app is made of, where it runs, how many databases there are, and how a change gets from here to members' phones. In plain words, with the real numbers.

1	2	35	~1,800
web app, for phones, iPads and computers	databases: test and live, the same shape	tables in each database	automatic checks before each release

In one line: TeeMate is a single website written in plain JavaScript (no app store, no framework). It keeps all the club's data in a Postgres database run by Supabase, and is hosted for free on GitHub Pages. There are two copies of the database: a **test** one to try changes on, and the **live** one members use.

The pieces and how they connect

Members' phones
The app in the web browser, or added to the home screen like an app. Works on iPhone and Android.

The club office iPad
The same app, signed in with the office login: it opens on the office pages.

The website (GitHub Pages)
The app's files: one page, the JavaScript, the styles and the course pictures. Free hosting. Address: gteemate.github.io/teemate

Supabase (live)
The database with all the club's data, sign-in, private photo storage, and the paper-scorecard reader.

Supabase (test)
An identical copy with made-up data, where every change is tried first. Members never see it.

Claude (Anthropic)
Reads paper scorecards: only the cropped score grid is sent, never names.

Open-Meteo
The weather at the course on Home. Free.

OpenStreetMap
Map tiles for finding the course's location. Free.

The phone talks to Supabase directly, with every request checked by the database's own rules (below). The website itself holds no data: it's the same files for every club member.

What it's written in

Part	Language	Size	What it is
The app	JavaScript (plain, no framework)	9,658 lines · 107 files	Every screen (43 member screens, 7 club office pages), scoring rules, games, knockouts, leagues, the paper-card flow
Look and feel	CSS	973 lines	One stylesheet; the club's colours come from the database
The page	HTML	1 file	One page; the JavaScript draws every screen into it
The database	SQL (Postgres)	82 changes · 5,533 lines	The tables, the rules for who may see and change what, and 130 functions such as booking a tee time or making a draw
The paper-card reader	TypeScript	112 lines	The one piece that runs on Supabase's servers, holding the Claude key. The only TypeScript in the project

Tests	JavaScript and SQL	8,425 + 3,581 lines	1,185 app tests and 599 database checks (see the testing PDF)

Libraries used by the app: three, kept small on purpose: Supabase's client (talking to the database), Leaflet (the map for the course location) and a QR-code maker (sharing your friend card). **Tools only on this computer:** Vite (bundles the files for the website), Vitest and happy-dom (the quick tests), and Playwright (the real-browser checks and printing these PDFs).

The two databases

Test (“teemate-test”)

- Made-up club data that can be wiped and reloaded at any time (the seed and scenarios).
- Every database change goes here first, and its database checks are run.
- Where the real-browser screen checks run.

Live

- What members use: today 139 members, 1,010 bookings, 130 scorecards, 6 events.
- Changes only after they pass on test, and only with your go-ahead.
- The demo logins (live@, admin@ and the rest, password “password”) live here too.

Both have the same 35 tables, kept in step by the numbered change files in `supabase/migrations`: each is applied once, to test then live, and recorded so it’s never applied twice. The seed and scenario scripts refuse to run on live.

The main groups of tables

Group	What it holds
Members and the club	Members, the office login, club settings and colours, friends, favourites, contacts from other clubs, game preferences
The course	Holes, tees, today’s pins
Tee times	Tee times, bookings and who’s on them, guest visits and points, requests, booking notices
Scorecards	Cards (one per group, shared), what each card counts for (events, leagues), paper-card reads
Competitions	Events (one-offs, seasons, leagues), templates, sign-up competitions and their entries, knockout draws
Club life	The halfway hut menu and orders, challenges between groups, balances

Keeping data safe

- **The database decides, not the app.** 47 row-level rules say who can see and change each row (a member sees their own requests; only players on a card can change it; only the office can delete a member). The app can’t get round them, even if someone tampers with it.
- **Anything with rules goes through a database function** (130 of them): booking checks the two-hour gap and the release time; entering checks the closing date and places. Each refusal comes back as a plain message the app shows.
- **Secrets stay on the server.** The phone only has the public key (which can do nothing the rules don’t allow). The Claude key is a Supabase secret, never in the app or the code.
- **Paper-card photos** are cropped to the score grid on the phone first (names never leave it), kept in a private store that only the players on that card and the office can open.

Speed: what’s kept on the phone

Lists that rarely change (members, the course, events, competitions, club settings) are kept on the phone, so screens open instantly. The database stamps each kind of data when it changes; the phone checks the stamps and only fetches what’s changed. Tee times, scorecards and alerts are always fetched fresh. A small service worker keeps the app’s own files, so it opens quickly even on a poor signal.

How a change reaches members

1. Build it
On this computer, with a test written first for each change

›

2. Check it
1,185 app tests, including every journey and every refusal

›

3. Test database
Database changes applied to test; 599 database checks

›

4. Your go-ahead
Nothing goes live without it

›

5. Live database
The same change, then the checks again on live

›

6. Push
GitHub runs every test again, builds the site and publishes it: live in about a minute

Members get the new version the next time they open or refresh the app. If the tests fail on GitHub, nothing is published.

How the code is organised

Folder	What's in it
`src/screens`	The member screens: Home, booking, the scorecard, competitions, knockouts, leagues, the course guide, friends, account
`src/office`	The club office pages: Today, Members, Tee sheet, Leaderboard, Competitions, Club
`src/api` and `src/api.js`	Every call to the database goes through one front door, so the backend could be swapped (to AWS, say) without touching the screens
`src/*.js`	The rules that don't need the database: scoring, handicaps and shots, every game, match play, knockout brackets, release times, weather
`src/paper`	Paper scorecards: the crop box, reading the result, the flow
`supabase/`	The database changes, the database checks, and the paper-card reader function
`src/test`	The journeys, the pretend server they run on, the feature register and the testing report
`scripts/`	Commands: run database changes and checks, load test data, make the testing PDF

Running costs

Service	What for	Cost now
GitHub Pages	Hosting the website	Free
Supabase	Two databases, sign-in, photo storage, the reader function	Free plan, or Pro (about $25 a month per project) for daily backups and more room as clubs join: your plan shows in the Supabase dashboard
Anthropic (Claude)	Reading paper scorecards	Pay as you go, about 1–2p per card, with a monthly cap you set
Open-Meteo, OpenStreetMap	Weather, map tiles	Free

Numbers counted from the code and the databases on 27 October 2026. See also: TeeMate-testing.pdf (what's tested and how it went).